
UNIVERSITÉ PARIS NANTERRE

Master 1 MIAGE

Optimisation combinatoire

GraphBench Challenge

Refutation automatique de conjectures en théorie des graphes

Binôme : Seyed Amineddin MIRI et Marie BIBI
Enseignement : Optimisation combinatoire
Enseignant : François DELBOT
Année universitaire : 2025–2026
Date de rendu : 7 mai 2026
Dépôt GitHub : <https://github.com/Aminmiri82/projet-optimisation-combinatoire>

Conjectures réfutées : 100/100
Temps total Partie 1 : 136,80 s
Temps moyen Partie 1 : 1,368 s
FunSearch (GPT-5,5, `candidate_006`) : 100/100
FunSearch (DeepSeek, `candidate_014`) : 100/100

Table des matières

1	Introduction et objectif	2
2	Heuristique simple	2
2.1	Principe de validation	2
2.2	Boucle de recherche	2
2.3	Evolution de <code>generate_initial_population</code>	3
2.4	Score manuel	5
3	Architecture FunSearch	5
3.1	Analyse du candidat GPT-5.5 <code>candidate_006</code>	5
3.2	Analyse du candidat DeepSeek <code>candidate_014</code>	6
4	Resultats experimentaux	6
5	Discussion scientifique	7
6	Dependances et bibliotheques	7

1 Introduction et objectif

Le projet GraphBench demande de réfuter automatiquement des conjectures de théorie des graphes. Une conjecture compare deux invariants d'un graphe, par exemple

$$y(G) \leq f(x(G)) \quad \text{ou} \quad y(G) \geq f(x(G)).$$

Un contre-exemple est un graphe appartenant aux classes imposées par le benchmark, pour lequel l'inégalité est strictement violée. Dans le benchmark utilisé, les classes principales sont **connected**, **tree** et **claw_free**.

Le système doit donc charger les conjectures, générer des graphes admissibles, calculer les invariants requis, maximiser la violation, puis produire un graphe au format **graph6**. Notre solution se décompose en deux parties. La Partie 1 est une recherche heuristique conçue à la main. La Partie 2 implémente une boucle de type FunSearch ou un modèle de langage propose des fonctions de score, qui sont ensuite évaluées automatiquement.

Le résultat principal est le suivant : la Partie 1 atteint 100/100 contre-exemples valides avec un coût total de 136.801705 secondes, soit 1.368017 seconde en moyenne. FunSearch atteint également 100/100 avec certains candidats, mais ne dépasse pas le temps de l'heuristique manuelle finale.

2 Heuristique simple

2.1 Principe de validation

La validation finale utilise uniquement la violation mathématique de la conjecture. Pour une conjecture $y \leq f(x)$, la violation est

$$\text{violation}(G) = y(G) - f(x(G)).$$

Pour une conjecture $y \geq f(x)$, elle est

$$\text{violation}(G) = f(x(G)) - y(G).$$

Un graphe est accepté seulement si la violation est strictement positive et si les contraintes de classe sont vérifiées après recalcul des invariants. Cette séparation est importante : le score heuristique peut guider la recherche, mais il ne peut pas valider un faux contre-exemple.

Module	Role
<code>loader.py</code>	Chargement du benchmark CSV
<code>conjecture.py</code>	Objet conjecture et calcul exact de la violation
<code>invariants.py</code>	Calcul des invariants avec networkx et numpy
<code>classes.py</code>	Tests connected , tree , claw_free
<code>generators.py</code>	Génération initiale et familles de graphes
<code>mutations.py</code>	Modifications locales de graphes
<code>repair.py</code>	Réparation après génération ou mutation
<code>search.py</code>	Sélection, mutation, évaluation et arrêt
<code>verify.py</code>	Vérification stricte des graphes graph6

TABLE 1 – Architecture de la Partie 1.

2.2 Boucle de recherche

La fonction principale conserve une population de graphes scores. Elle commence par une population initiale, sélectionne ensuite des parents prometteurs, applique une mutation locale ou injecte parfois

un graphe frais, repare le graphe si necessaire, puis calcule seulement les invariants utiles a la conjecture courante. Les candidats sont tries par score objectif, mais la condition d'arret reste la validation stricte.

Le coeur de l'evaluation est volontairement robuste aux scores generes automatiquement. Si un score leve une exception, retourne `None` ou produit une valeur non finie, l'objectif devient $-\infty$, mais le moteur continue.

Listing 1 – Evaluation d'un candidat dans le moteur de recherche.

```
def _evaluate(graph, conjecture, needed, scorer):
    invariants = compute_cached(graph, needed)
    try:
        raw_objective = scorer(graph, invariants, conjecture)
        objective = float(raw_objective) if raw_objective is not None else float("-inf")
    except Exception:
        objective = float("-inf")
    if not math.isfinite(objective):
        objective = float("-inf")
    return objective, violation_score(graph, invariants, conjecture), invariants
```

2.3 Evolution de generate_initial_population

L'amelioration la plus decisive de la Partie 1 n'a pas ete le score seul, mais l'espace de graphes donne au moteur. Au depart, `generate_initial_population` ne faisait qu'echantillonner aleatoirement des graphes selon la classe demandee. Cette approche explore, mais elle manque les structures extremales rares.

Listing 2 – Version v1 : population aleatoire reparee.

```
population = []
while len(population) < size and attempts < size * 80:
    n = rng.randint(min_order, max_order)
    graph = generate_graph(classes, rng, n)
    graph = repair(graph, classes, rng)
    if graph is not None and satisfies_classes(graph, classes):
        population.append(_normalize_nodes(graph))
```

En v2, nous avons ajoute une phase deterministe avant l'echantillonnage aleatoire. La population commence par des graphes simples et quelques familles cibles. Ce changement transforme la recherche : au lieu d'attendre qu'une mutation aleatoire tombe sur une bonne forme, le moteur dispose immediatement de contre-exemples proches.

Listing 3 – Version v2 et suivantes : graines structurelles avant le hasard.

```
for graph in seed_graphs(classes, min_order, max_order):
    graph = repair(graph, classes, rng)
    if graph is not None and satisfies_classes(graph, classes):
        population.append(_normalize_nodes(graph))
if len(population) >= size:
    return population
```

Les premieres graines etaient des chemins, cycles, cliques et araignees. Les araignees ont ete ajoutees car elles augmentent certains invariants de domination sans augmenter aussi vite le couplage. Elles sont construites a partir d'un centre et de plusieurs pattes de longueurs controlees.

Listing 4 – Araignees introduites en v2.

```
def _spider(lengths):
    graph = nx.Graph()
```

```

graph.add_node(0)
next_node = 1
for length in lengths:
    previous = 0
    for _ in range(length):
        graph.add_edge(previous, next_node)
        previous = next_node
        next_node += 1
return graph

```

En v3, la population initiale s'est elargie avec des doubles etoiles, des chaines de cliques endommagees, des graphes connexes clairsemes, des triangles avec chemins attaches et des graphes clique–chemin–clique. Les doubles etoiles ciblent les cas ou l'on veut beaucoup de feuilles autour de peu de sommets centraux. Les chaines de cliques endommagees donnent une densite et des triangles eleves tout en modifiant le degre maximal ou la connectivite. Les graphes clique–chemin–clique sont utiles pour des conjectures ou les grandeurs spectrales et les distances reagissent fortement a un goulot d'etranglement.

Listing 5 – Exemple de graine clique–chemin–clique.

```

def _clique_path_clique(left_size, right_size, path_length):
    left = list(range(left_size))
    path = list(range(left_size, left_size + path_length))
    right = list(range(left_size + path_length,
                       left_size + path_length + right_size))
    graph.add_edges_from((u, v) for i, u in enumerate(left)
                          for v in left[i + 1:])
    graph.add_edges_from((u, v) for i, u in enumerate(right)
                          for v in right[i + 1:])
    previous = left[0]
    for node in path:
        graph.add_edge(previous, node)
        previous = node
    graph.add_edge(previous, right[0])

```

La v4 a stabilise les derniers echecs en ajoutant les chemins branches et les queues triangulaires. Les chemins branches gardent une structure d'arbre, mais creent localement des effets forts sur la domination independante, les feuilles et certains indices de Zagreb. Les queues triangulaires conservent une structure proche du sans-griffe tout en controlant les distances.

Version	Ajouts dans la population initiale	Motivation
v1	Generation aleatoire par classe : arbres, chemins, cycles, cliques, $G(n, p)$, graphes denses sans griffe.	Obtenir une base modulaire et verifier le pipeline.
v2	Graines deterministes, chemins/cycles/cliques de tailles fixes, araignees.	Donner au moteur des graphes extremaux simples des le debut.
v3	Doubles etoiles, cliques endommagees, graphes clairsemes, triangles avec chemins, clique–chemin–clique.	Cibler domination, densite, triangles, distances et effets spectraux.
v4	Chemins branches et queues triangulaires.	Resoudre les derniers cas d'arbres, de graphes sans griffe et de distances spectrales.

TABLE 2 – Evolution de `generate_initial_population`.

2.4 Score manuel

Le score manuel part de la violation brute et ajoute de faibles termes de guidage normalises. Le coefficient dominant reste la violation, afin de ne pas éloigner la recherche de l'objectif mathématique.

Listing 6 – Forme simplifiée du score manuel.

```
score = 100.0 * violation
score += 0.15 * y_value if conjecture.sign == "<=" else -0.15 * y_value
score += 0.12 * clamp(direction_from_derivative) * x_value
score += structural_guidance(G, invariants, conjecture)
```

Les termes structurels regardent la densité, la proportion de feuilles, les triangles et le diamètre. Ils favorisent les formes cohérentes avec l'invariant que l'on veut rendre grand ou petit. Cependant, les expériences montrent que ces termes ont surtout servi à départager des candidats proches ; le saut de performance vient principalement des familles de graphes initiales.

3 Architecture FunSearch

La Partie 2 implémente une boucle inspirée de FunSearch. Le modèle génère une fonction Python de score, nommée `heuristic_score`. Elle est sauvegardée dans `src/graphbench/funsearch/candidates/`, puis évaluée par le même moteur que la Partie 1. Seul le score change ; les générateurs, mutations, invariants et vérifications restent identiques.

Le cycle est le suivant :

1. lire le registre des candidats déjà évalués ;
2. sélectionner les meilleurs fichiers sources ;
3. construire un prompt avec ces sources et leurs résultats ;
4. appeler OpenRouter pour obtenir une nouvelle fonction Python ;
5. extraire et valider localement la fonction ;
6. lancer l'évaluateur et ajouter une ligne dans `results/funsearch/registry.csv`.

Nous avons d'abord utilisé GPT-5.5 avec raisonnement faible, car il était attendu comme fort pour la génération de code et d'heuristiques, tout en gardant un coût inférieur aux modes de raisonnement plus élevés. Ensuite, pour une séquence plus longue de 25 générations, nous sommes passés à DeepSeek pour des raisons de coût.

Le commit 7527406 correspond à l'expérience GPT-5.5 initiale. Son registre montre six générations principales : `candidate_001` et `candidate_002` trouvent 29/30 sur une évaluation courte, `candidate_003` trouve 28/30, `candidate_004` trouve 97/100, `candidate_005` trouve 98/100, et `candidate_006` ne trouve d'abord que 95/100 dans une ligne du registre. Après réévaluation complète dans `candidate_006_100.csv`, `candidate_006` est le seul candidat de cette série à atteindre 100/100.

3.1 Analyse du candidat GPT-5.5 candidate_006

Le candidat 006 reste proche du score manuel, mais il introduit une idée intéressante : donner un bonus aux graphes proches de la frontière de décision. Il multiplie la violation par 50, puis ajoute un terme gaussien maximal lorsque la violation est proche de zéro.

Listing 7 – Extrait de `candidate_006.py`.

```
score = 50.0 * violation
boundary_weight = math.exp(-5.0 * violation * violation)
diff_ratio = diff_abs / (diff_abs + n + 1.0)
score += 3.0 * boundary_weight * diff_ratio
```

Ce terme est raisonnable pour une recherche locale : un graphe presque contre-exemple est souvent plus utile comme parent qu'un graphe tres loin de la frontiere. Le candidat ajoute aussi une direction derivee du polynome f , utilisee surtout quand la violation est encore negative.

Listing 8 – Guidage par derivee dans `candidate_006.py`.

```
if sign == '<=':
    deriv_bonus = deriv / (abs(deriv) + 1.0)
else:
    deriv_bonus = -deriv / (abs(deriv) + 1.0)
if violation < 0:
    score += 0.5 * deriv_bonus
```

Enfin, il contient des indices faibles pour les arbres et les graphes sans griffe. Pour les arbres, il favorise les feuilles ; pour les graphes sans griffe, il penalise legerement un degre maximal trop eleve. Ces choix ne suffisent pas a battre la Partie 1, mais ils produisent un score stable et compatible avec le moteur.

3.2 Analyse du candidat DeepSeek candidate_014

Dans la sequence DeepSeek de 25 cycles, `candidate_014` est le meilleur candidat du registre. Il reprend presque la meme structure que 006, mais avec une frontiere un peu plus accentuee et une penalite explicite si la direction derivee va contre l'objectif.

Listing 9 – Extrait de `candidate_014.py`.

```
boundary_weight = math.exp(-6.0 * violation * violation)
diff_ratio = diff_abs / (diff_abs + n + 1.0)
score += 3.5 * boundary_weight * diff_ratio

if sign == '<=':
    direction = deriv
else:
    direction = -deriv
dir_norm = direction / (abs(deriv) + 1.0)
away_penalty = max(0.0, -dir_norm) * 0.3
if violation < 0:
    score += 0.5 * dir_norm - away_penalty
```

Scientifiquement, ce candidat est coherent : il conserve la violation comme signal principal, encourage les points proches de la frontiere et evite certaines directions localement defavorables. Son resultat complet est 100/100, avec un cout total de 166.636643 secondes. Il est donc valide mais plus lent que l'heuristique manuelle et que le candidat GPT-5.5 006.

4 Resultats experimentaux

Methode	Trouve	Cout total	Temps moyen
Heuristique manuelle Partie 1	100/100	136.80	1.368 s
FunSearch GPT-5.5, candidat 006	100/100	157.50	1.575 s
FunSearch DeepSeek, candidat 014	100/100	166.64	1.666 s

TABLE 3 – Resultats finaux sur les 100 conjectures.

La Partie 1 est la meilleure en temps moyen. Elle obtient une mediane de 0.561114 seconde et un maximum de 5.347417 secondes. Le candidat GPT-5.5 006 atteint 100/100 avec un cout total de

157.502736 secondes. Le candidat DeepSeek 014 atteint 100/100 avec un cout total de 166.636643 secondes et constitue le meilleur candidat de la sequence DeepSeek.

Le fait que FunSearch atteigne 100/100 est important : l'architecture produit des fonctions executables et valides. En revanche, elle n'ameliore pas le temps final. L'explication la plus probable est que le moteur de Partie 1 etait deja tres fortement aide par ses familles initiales. Lorsque le bon graphe est deja present dans la population initiale ou proche d'elle, la qualite fine du score devient secondaire.

5 Discussion scientifique

Les conjectures faciles etaient celles pour lesquelles une famille extremale simple existait : petits graphes complets pour certains cas de clique ou de degre moyen, chemins pour certains cas de distance, araignees pour domination totale contre couplage, doubles etoiles pour domination independante et couverture.

Les conjectures difficiles combinaient souvent des invariants couteux ou sensibles : distance, valeurs propres, domination, domination totale et domination independante. Les cas spectraux incluient notamment la plus grande valeur propre de distance et la deuxieme plus petite valeur propre du Laplacien. La correction de **proximity** et **remoteness** a ete un point cle : le benchmark utilise les conventions reciproques

$$\text{proximity} = 1/\max \bar{d}, \quad \text{remoteness} = 1/\min \bar{d}.$$

Avant cette correction, plusieurs cas semblaient difficiles pour de mauvaises raisons.

L'enseignement principal est que la generation de familles de graphes a compte davantage que le faconnage du score. Les mutations locales explorent efficacement autour d'une bonne forme, mais elles decouvrent mal certaines structures rares. En ajoutant les bonnes graines, on change la topologie de l'espace de recherche : le moteur commence pres des regions ou la violation positive existe.

L'IA a ete utile pour proposer des fonctions de score, mettre en place la boucle FunSearch et explorer des variantes. Elle a ete moins decisive pour identifier les familles extremales et pour comprendre les conventions specifiques du benchmark. L'intervention humaine est restee necessaire pour verifier les invariants, interpreter les echecs, choisir les graines et maintenir une validation stricte.

6 Dependances et bibliotheques

La solution est volontairement legere. Le fichier **requirements.txt** contient :

Dependance	Utilisation
networkx >= 3.2	Representation des graphes, generateurs, connectivite, line graphs, graph6
numpy >= 1.26	Calcul numerique, notamment pour les invariants spectraux

TABLE 4 – Dependances Python directes.

La bibliotheque standard Python est utilisee pour les fichiers CSV, les chemins, le temps, l'import dynamique de candidats FunSearch, les nombres aleatoires reproductibles et la gestion JSON. Pour la Partie 2, OpenRouter est appele via un client local du depot ; la cle API n'est pas versionnee. Le rapport est ecrit en LaTeX et compile avec **pdflatex**.